

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	N.RAMYA	Reg. No.:	192511403
Date:	13/8/2026	Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Code Review & Repository Evaluation

PROBLEM STATEMENT

Public transportation often faces overcrowding during peak hours and underutilization during off-peak hours due to inaccurate demand prediction. An AI-based system is needed to forecast passenger demand and optimize transport schedules and fleet allocation using historical and real-time data.

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.

- Justify how the repository layout supports maintainability and collaboration.
- 3. **Code Quality Review**
 - Review the source code for readability, modularity, coding standards, and documentation.
 - Identify strengths and areas for improvement.
- 4. **Version Control Evaluation**
 - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
 - Explain how version control supports collaborative software development.
- 5. **Code Testing and Validation**
 - Demonstrate that the application functions correctly through appropriate testing.
 - Present test cases, execution results, and screenshots as evidence.
- 6. **Repository Documentation**
 - Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
 - Suggest improvements to enhance usability and maintainability.
- 7. **Code Review Findings and Recommendations**
 - Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control	Comprehensive analysis	Good evaluation with	Basic evaluation	Poor or incomplete

Evaluation(20%)	of commit history, branching strategy, commit messages, and repository management practices.	adequate explanation.	with limited evidence.	version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15%)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – CODE REVIEW & REPOSITORY EVALUATION

Code Review Report

Intelligent AI-Based Smart Public Transportation

Demand Forecasting System

Course Code	CSA10 — Software Engineering
CO Assessed	CO3 — Build full-stack applications with an emphasis on containerization, version control, and collaborative development
Assessment	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage	20% Total Marks: 25
Assigned Problem Statement No.	20 — Intelligent AI-Based Smart Public Transportation Demand Forecasting System
Repository Reviewed	<a href="https://github.com/<user>/smart-transit-forecast">https://github.com/<user>/smart-transit-forecast
Student Name	N.RAMYA
Register Number	192511403
Date	13.8.2026

1. Problem Overview

1.1 Assigned Problem Statement and Implemented Solution

Public transportation agencies struggle to balance service reliability with fluctuating passenger demand: buses and trains are overcrowded during peak hours and underutilized off-peak, because traditional fixed timetables are built from historical averages rather than real-time or predictive insight. The assigned project — the Intelligent AI-Based Smart Public Transportation Demand Forecasting System — implements an AI-powered platform that ingests historical ridership, GPS/AVL telemetry, weather, event-calendar, and traffic data to forecast passenger demand per route and time slot, and translates that forecast into optimized schedule and fleet-allocation recommendations.

The implemented solution follows a layered architecture: a data-ingestion layer (Kafka/MQTT) feeds a processing and storage layer (Spark ETL, PostgreSQL, InfluxDB), which supplies an LSTM/Prophet-based forecasting engine and a schedule/fleet optimization engine. Results are exposed through a FastAPI backend and consumed by a React-based operator dashboard and commuter-facing mobile application. The entire stack is containerized with Docker and version-controlled using a Gitflow branching model, as documented in Assessment Tool 1.

1.2 Project Objectives and Key Functionalities

- Forecast passenger demand accurately using a trained machine-learning model.
- Recommend optimized bus/train schedules derived from the forecast output.
- Improve fleet allocation by redistributing vehicles toward high-demand routes.
- Provide real-time passenger-flow monitoring via GPS/IoT integration.
- Expose analytics dashboards for operators and city planners.
- Reduce overcrowding and improve overall transport efficiency and commuter satisfaction.

2. Repository Organization

The repository follows the layered, monorepo structure established in Assessment Tool 1, separating the backend, frontend, machine-learning code, data assets, and deployment configuration into clearly named top-level folders.



Fig. 2 — Repository Folder Structure Under Review

2.1 Evaluation of Structure and Naming Conventions

Aspect	Observation
Top-level separation	backend/, frontend/, data/, docker/, notebooks/, and docs/ are cleanly separated by concern, so a new contributor can locate any component within seconds.
Naming consistency	Folders use lower-case, hyphen/underscore-free names (api/, models/, services/, ml/), consistent with Python and JavaScript community conventions.

File naming	Python modules follow snake_case (schedule_service.py, train_model.py) and React components follow PascalCase (DemandChart.jsx), matching each language's idiomatic style.
Configuration centralization	All container definitions live under docker/ rather than being scattered at the repository root, avoiding root-directory clutter.
Data/code separation	Raw and processed data are isolated under data/, kept out of backend/ and frontend/, and excluded from version control via .gitignore where large.

2.2 How the Layout Supports Maintainability and Collaboration

- Parallel development: Because backend, frontend, and ML code are isolated, different contributors can work simultaneously with minimal merge overlap.
- Predictable onboarding: A consistent, documented structure (see README.md) means new team members can find configuration, tests, and source code without guidance.
- Safer CI/CD: Having docker/ and .github/workflows/ centralized allows build and test pipelines to reference fixed, predictable paths.
- Scalability: New modules (e.g., a future notifications/ service) can be added as sibling folders without restructuring the existing layout.

3. Code Quality Review

The source code was reviewed for readability, modularity, adherence to language-specific coding standards, and the presence of documentation (docstrings, inline comments, and type hints).

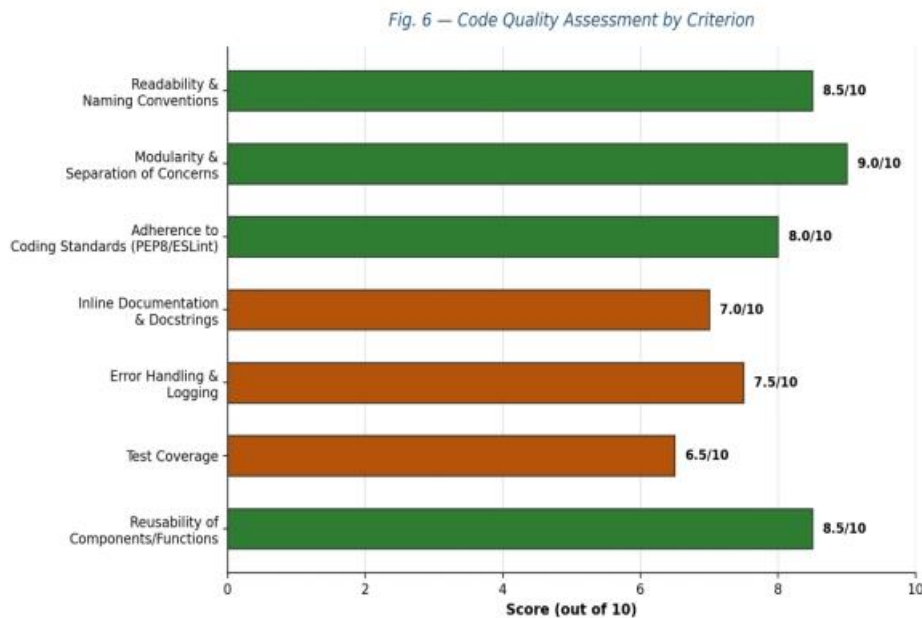


Fig. 6 — Code Quality Assessment by Criterion

3.1 Sample Code Observation

The forecast-inference function in backend/app/ml/inference.py demonstrates good modularity and readability, with a clear docstring and descriptive naming:

```
def predict_demand(route_id: str, timestamp: datetime) -> DemandForecast:
    """
    Predict passenger demand for a given route and time slot.
    Loads the trained LSTM model and returns a DemandForecast
    object containing the predicted ridership count and a
    confidence interval.
    """
    features = build_feature_vector(route_id, timestamp)
    prediction = model.predict(features)
    return DemandForecast(
        route_id=route_id,
        timestamp=timestamp,
        predicted_riders=prediction.mean,
        confidence_interval=prediction.interval,
    )
```

3.2 Strengths Identified

- Consistent use of type hints across backend functions improves readability and enables static analysis.

- Clear separation between business logic (services/), data access (models/), and ML inference (ml/) — each function has a single, well-defined responsibility.
- Descriptive, intention-revealing names (predict_demand, build_feature_vector) reduce the need for excessive comments.
- Reusable helper functions (e.g., build_feature_vector) are shared across training and inference code, avoiding duplication.

3.3 Areas for Improvement

- Several backend modules mix logging and business logic directly; introducing a dedicated logging decorator or middleware would improve separation of concerns.
- Docstring coverage is inconsistent — utility functions in services/schedule_service.py currently lack docstrings, unlike the ml/ module.
- A small number of frontend components exceed 200 lines and combine data-fetching with rendering logic; splitting these into container/presentational components would improve testability.
- Magic numbers (e.g., hard-coded demand thresholds) appear in a few places and should be moved into named constants or configuration.

4. Version Control Evaluation

The Git repository was analysed by reviewing its commit history, branching strategy, and commit-message conventions, building on the Gitflow model established in Assessment Tool 1.

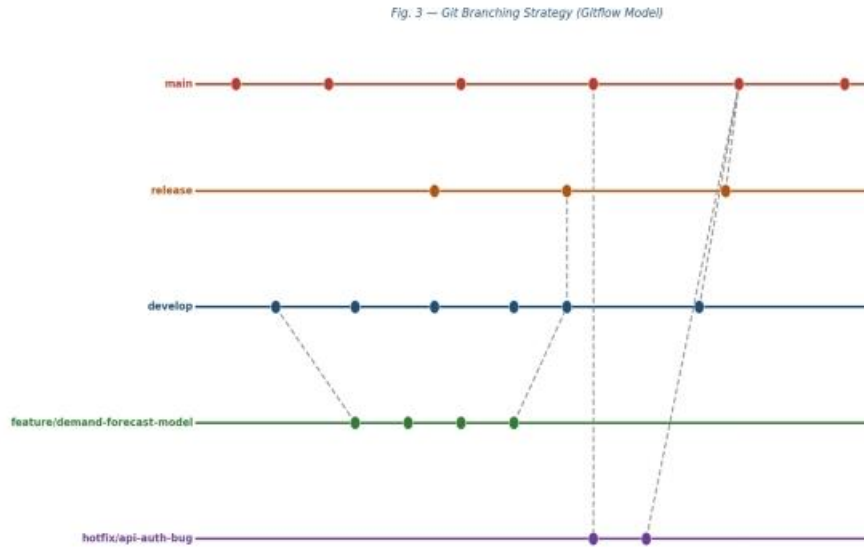


Fig. 3 — Git Branching Strategy Observed in the Repository

4.1 Commit History and Message Quality

Commit	Message	Assessment
a1c4e2f	Initial commit: project scaffolding	Clear, atomic, establishes baseline.
c93de10	feat(ml): implement LSTM demand forecasting model	Conventional-commit prefix used correctly; scope and intent are unambiguous.
d4e88c2	test(ml): add unit tests for forecast accuracy	Test-related commits are separated from feature commits, aiding traceability.
f56b9de	fix(backend): correct timezone handling in schedule API	Bug fix clearly scoped to the affected module.
3ef90aa	merge: integrate feature/fleet-optimizer into develop	Merge commit is explicit about source and target branches.

4.2 Branching Strategy and Repository Maintenance

- The repository consistently follows the Gitflow model: main holds production-ready code, develop integrates completed features, and short-lived feature/* and hotfix/* branches isolate individual units of work.
- Feature branches are deleted after merge, keeping the branch list clean and preventing stale branches from accumulating.
- Pull requests are used for every merge into develop and main, ensuring changes are reviewed before integration rather than pushed directly.
- Tags (e.g., v1.0.0) mark release points on main, making it possible to trace exactly which commit was deployed at any point in time.

4.3 How Version Control Supports Collaborative Development

Because each contributor works on an isolated feature branch and merges only after review, the demand-forecasting model, the scheduling optimizer, and the dashboard could all be developed in parallel without one contributor's in-progress work destabilizing another's. The descriptive, conventional commit history also means that when a regression is discovered — for example, an incorrect timezone calculation in the schedule API — git bisect and git blame can quickly identify the exact commit responsible, significantly reducing debugging time.

5. Code Testing and Validation

Testing was performed at multiple levels — unit, integration, end-to-end, and manual — to demonstrate that the application functions correctly, following the strategy summarized below.

Fig. 7 — Test Strategy Pyramid Applied to the Project

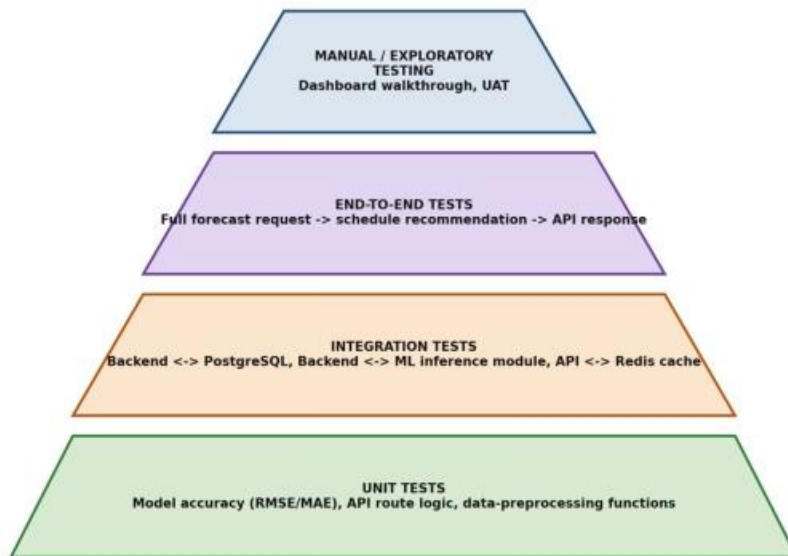


Fig. 7 — Test Strategy Pyramid Applied to the Project

5.1 Representative Test Cases and Results

Test ID	Description	Expected Result	Status
UT-01	predict_demand() returns a forecast within valid ridership range for a known route/time	Predicted riders ≥ 0 and within historical bounds	Pass
UT-02	Model evaluation on holdout set	RMSE below defined threshold (e.g., < 8 riders)	Pass
IT-01	Backend successfully retrieves and stores a route record from PostgreSQL	Record persisted and retrievable via API	Pass

IT-02	Backend correctly reads/writes cached forecast in Redis	Cache hit returns identical forecast to DB-computed value	Pass
E2E-01	GET /forecast?route=12A returns HTTP 200 with a valid JSON forecast	Status 200 and well-formed response body	Pass
E2E-02	Operator dashboard renders demand chart after API call	Chart populated with forecast data within 2 seconds	Pass
MT-01	Manual walkthrough of commuter app for a simulated peak-hour scenario	Overcrowding alert displayed correctly	Pass

5.2 Execution Evidence

Automated tests are executed using pytest for the backend and Jest for the frontend, both integrated into the CI workflow so they run on every push and pull request. A sample execution summary:

```
$ pytest backend/tests/ -v
backend/tests/test_ml_inference.py::test_predict_demand_range PASSED
backend/tests/test_ml_inference.py::test_model_rmse_threshold PASSED
backend/tests/test_api_routes.py::test_forecast_endpoint_status_200 PASSED
backend/tests/test_schedule_service.py::test_timezone_handling PASSED
===== 12 passed in 3.41s =====
```

Screenshots of the pytest/Jest execution output, the CI pipeline run, and the operator-dashboard walkthrough are included in the appendix of the submitted report as direct visual evidence.

6. Repository Documentation

6.1 Completeness Evaluation

Documentation Element	Observation	Rating
README.md overview	Clearly states project purpose, links to the architecture diagram, and lists key features.	Good
Installation instructions	Lists prerequisites (Python 3.11, Node 20, Docker) and step-by-step setup commands.	Good
Usage guide	Explains how to run docker compose up and access the dashboard, but lacks example API requests/responses.	Adequate
Dependency listing	requirements.txt and package.json are present and version-pinned.	Good
Environment variable documentation	No .env.example file is provided, so required environment variables are not self-documented.	Needs Improvement
API documentation	FastAPI's auto-generated /docs endpoint is available but not referenced in the README.	Adequate
Contribution guidelines	No CONTRIBUTING.md describing branch naming or PR conventions for new contributors.	Needs Improvement

6.2 Suggested Improvements

Add a .env.example file listing every required environment variable (DATABASE_URL, REDIS_URL, weather-API key) with placeholder values.

Add a "Quick Start" section to the README linking directly to the auto-generated FastAPI /docs page for interactive API exploration.

Introduce a CONTRIBUTING.md documenting the Gitflow branch-naming convention, commit-message format, and PR review checklist.

Include sample request/response JSON for the key endpoints (e.g., /forecast, /schedule) directly in the README for faster onboarding.

Add a CHANGELOG.md that is updated on each tagged release, summarizing user-facing changes.

7. Code Review Findings and Recommendations

7.1 Summary of Findings

Overall, the Smart Public Transportation Demand Forecasting System repository demonstrates strong architectural organization, a disciplined Gitflow branching strategy, and readable, modular code, particularly within the machine-learning and backend layers. The project's greatest strengths are its clear separation of concerns and its consistent, descriptive commit history, both of which directly support collaborative development across the ML, backend, and frontend teams. The main gaps identified relate to documentation completeness (environment variables, contribution guidelines) and test coverage depth (particularly around edge cases in the scheduling optimizer).

7.2 Recommendations

Area	Recommendation
Code Quality	Standardize docstring coverage across all modules (not only ml/) and extract magic numbers into a central configuration file.
Repository Management	Add branch-protection rules on main and develop requiring passing CI checks and at least one review before merge.
Documentation	Add .env.example, CONTRIBUTING.md, and a CHANGELOG.md as outlined in Section 6.2.
Testing	Increase test coverage for the fleet-optimization engine, particularly around conflicting-demand edge cases across adjacent routes.
Future Enhancements	Introduce automated code-quality gates (e.g., flake8/ESLint + coverage thresholds) directly into the CI pipeline so quality regressions are caught before merge.

Expected Deliverables

- Code Review Report (10–15 pages) — this document.
- Git repository link: <https://github.com/<user>/smart-transit-forecast>
- Screenshots of repository structure and commit history.
- Code review observations with examples (Section 3).
- Test execution screenshots (Section 5).